#1 Method `cost` takes the current solution, `P` (a graph embedding, and also a point in the problem space), width (`w`) and height (`h`) of the canvas, and the normalizing factors for the components.
#2 Cycle through the vertices in the graph.
#3 Add the second component of the objective function, corresponding to the repulsive force between vertices and the canvas' borders.
#4 Cycle through all vertices except v.
#5 Add the first component of the cost function, the repulsive force between vertices.
#6 Cycle through edges; for each edge `e` between vertices `u` and v…
#7 Add the third component, the attractive force between adjacent vertices.
#8 Cycle through all vertices except the edge's endpoints.
#9 Add the fifth component, the repulsive force between edges and vertices (except for each edge's endpoints, of course).
#10 At last, add the fourth component, proportional to the number of edge intersections in the embedding.

The algorithm in the paper was only using the vertex local update heuristic, but not with a constant range, rather making the neighborhood in which a vertex can be moved smaller with the progressing of the algorithm.

You can also find the working code implemented for *JsGraphs* library on GitHub.

Speaking of good embeddings, in this case it makes little sense to check the quality of the results by looking at the